SGDD

Sistema de Gestion de Dulcerias

PLAN MAESTRO DE PRUEBAS DE SOFTWARE (STP)

Version 1.0.0 | 2026

Autor: Bryan Stiven Bonilla Rosas

Plataforma SaaS Multi-Tenant | Cobertura minima: 80%

INFORMACION DEL DOCUMENTO

Campo	Detalle
Titulo	Plan Maestro de Pruebas de Software (STP)
Sistema	SGDD - Sistema de Gestion de Dulcerias
Version	1.0.0
Autor	Bryan Stiven Bonilla Rosas
Fecha	2026
Estado	Activo
Cobertura minima exigida	80% de codigo | 100% de casos criticos
Estandar de referencia	IEEE 829 - Standard for Software Test Documentation

Historial de Versiones

Version	Fecha	Autor	Cambio
1.0.0	2026	Bryan Stiven Bonilla Rosas	Creacion inicial del STP

1. INTRODUCCION

Este documento define el Plan Maestro de Pruebas (STP) del sistema SGDD, una plataforma SaaS multi-tenant para la gestion integral de dulcerias. El STP establece la estrategia, los tipos de pruebas, las herramientas, los criterios de aceptacion y el proceso de gestion de defectos que guiaran el aseguramiento de la calidad del sistema.

1.1 Proposito

El proposito de este documento es:

- Definir la estrategia global de pruebas del sistema SGDD.
- Establecer los tipos de pruebas aplicables y su cobertura minima.
- Documentar los ambientes, herramientas y responsabilidades del proceso de QA.
- Proveer trazabilidad entre requisitos funcionales y casos de prueba.
- Establecer los criterios de entrada, salida y aceptacion para cada fase de pruebas.

1.2 Alcance

El plan de pruebas cubre todos los modulos funcionales del sistema SGDD:

Modulo	Incluido en pruebas
Autenticacion y autorizacion (JWT + RBAC)	Si
Punto de Venta (POS) - Registro de ventas	Si
Control de inventario por lotes	Si
Gestion de clientes	Si
Reportes y analitica	Si
Prediccion de demanda (ML)	Si
Bot conversacional (NLP)	Si
Multi-tenancy e aislamiento de datos	Si - Critico
API REST - todos los endpoints	Si
DevOps / CI/CD pipeline	Si - Automatizado

2. ESTRATEGIA DE PRUEBAS

La estrategia de pruebas del sistema SGDD sigue el modelo de la piramide de pruebas, priorizando la automatizacion desde la base y complementando con pruebas de integracion y end-to-end en los niveles superiores.

2.1 Piramide de pruebas

```
          /\           /  \   E2E (Playwright)          /----\         /      \
Integracion (pytest + testcontainers)       /--------\      /          \
Unitarias (pytest + Jest)     /___________\
```

2.2 Niveles de prueba

Nivel	Herramienta	Cobertura objetivo	Ejecucion
Pruebas Unitarias	pytest / Jest	>=80% del codigo	En cada commit (CI)
Pruebas de Integracion	pytest + testcontainers	>=70% de flujos criticos	En cada PR a develop
Pruebas E2E	Playwright	100% de flujos criticos	En cada PR a main
Pruebas de Seguridad	OWASP ZAP / Bandit	100% de endpoints	Semanal / en releases
Pruebas de Rendimiento	Locust	Escenarios de carga	Antes de cada release
Pruebas de Regresion	Suite automatizada	100% de funciones previas	En cada release

2.3 Criterios de entrada y salida

Criterios de entrada (inicio de pruebas)

- El modulo a probar tiene codigo mergeado en la rama de pruebas.
- El ambiente de testing esta configurado y los contenedores Docker estan activos.
- Los casos de prueba para el modulo estan documentados y revisados.
- Los datos de prueba estan disponibles y aislados del entorno de produccion.

Criterios de salida (fin de pruebas)

- Cobertura de codigo >= 80% para el modulo.
- 0 defectos criticos o bloqueantes abiertos.
- Todos los casos de prueba de prioridad alta marcados como PASS.
- Reporte de pruebas generado y revisado por el equipo.

3. TIPOS DE PRUEBAS

3.1 Pruebas Unitarias

Validan el comportamiento de unidades individuales de codigo (funciones, clases, metodos) de forma aislada, usando mocks para dependencias externas.

Componente	Framework	Descripcion
Backend - Dominio	pytest	Entidades: Producto, Venta, Cliente, Inventario
Backend - Casos de Uso	pytest + mock	RegistrarVentaUseCase, ConsultarStockUseCase, etc.
Backend - Repositorios	pytest + mock	Repositorios SQLAlchemy con DB en memoria
Frontend - Componentes	Jest + RTL	Componentes React: POS, Formularios, Dashboards
Frontend - Hooks	Jest	Custom hooks de estado y llamadas a la API
Modelos ML	pytest	Funciones de preprocesamiento y evaluacion de metricas

```
# Ejemplo de prueba unitaria - Caso de uso RegistrarVenta def
test_registrar_venta_descuenta_inventario():     repo_mock = Mock()
repo_mock.obtener_stock.return_value = 10     use_case =
RegistrarVentaUseCase(repo_mock)     resultado =
use_case.ejecutar(producto_id=1, cantidad=3)     assert
resultado.stock_restante == 7
repo_mock.actualizar_stock.assert_called_once_with(1, 7)
```

3.2 Pruebas de Integracion

Verifican que los componentes del sistema interactuen correctamente entre si. Se utiliza una base de datos PostgreSQL real en contenedor Docker aislado.

Escenario de integracion	Descripcion
API + Base de datos	Los endpoints REST persisten y recuperan datos correctamente en PostgreSQL
API + Redis	El cache se puebla y se invalida correctamente en operaciones de escritura
Multi-tenancy	Un tenant no puede acceder a datos de otro tenant bajo ningun flujo
Autenticacion + RBAC	El middleware JWT bloquea rutas no autorizadas segun el rol del usuario
POS + Inventario	Al registrar una venta se descuenta el stock y se dispara alerta si llega al minimo
Prediccion ML + DB	El modelo genera forecast y lo persiste correctamente con tenant_id

```
# Ejemplo de prueba de integracion con TestClient de FastAPI def
test_post_venta_descuenta_stock(client, db_session):     headers =
{"Authorization": "Bearer " + get_test_token()}     payload = {"producto_id":
"uuid-1", "cantidad": 2}     response = client.post("/api/v1/ventas",
json=payload, headers=headers)     assert response.status_code == 201     stock
= db_session.query(Inventario).filter_by(producto_id="uuid-1").first()
assert stock.cantidad == stock_inicial - 2
```

3.3 Pruebas End-to-End (E2E)

Simulan el comportamiento del usuario final en el navegador usando Playwright. Cubren los flujos de negocio completos desde la interfaz hasta la base de datos.

Flujo E2E	Descripcion	Prioridad
Login exitoso	El usuario ingresa credenciales validas y accede al dashboard	Alta
Flujo de venta completo	Vendedor registra venta con 3 productos, genera recibo y actualiza stock	Alta - Critico
Alta de producto	Admin crea producto, lo asigna a categoria y verifica en inventario	Alta
Recepcion de lote	Operador registra entrada de mercancia y verifica fecha de vencimiento	Alta
Consulta al bot	Usuario pregunta en lenguaje natural y recibe respuesta con datos reales	Media
Reporte de ventas	Supervisor filtra reporte por fecha y exporta a Excel	Media
Prediccion de demanda	Admin visualiza forecast de los proximos 30 dias en dashboard	Media
Aislamiento multi-tenant	Tenant A no puede ver datos de Tenant B desde la UI	Alta - Critico

3.4 Pruebas de Seguridad

Prueba de seguridad	Herramienta	Descripcion
Escaneo OWASP Top 10	OWASP ZAP	Detecta vulnerabilidades en la API: SQL Injection, XSS, CSRF, etc.
Analisis estatico de codigo	Bandit	Detecta patrones inseguros en el codigo Python
Fuerza bruta al login	Locust	Verifica que el rate limiting bloquea correctamente
JWT invalido / expirado	pytest	La API rechaza tokens invalidos, expirados o con firma incorrecta
Acceso cruzado entre tenants	pytest	Un JWT de Tenant A no puede acceder a datos de Tenant B
Inyeccion SQL	OWASP ZAP	Los parametros de entrada no permiten inyeccion de SQL

3.5 Pruebas de Rendimiento

Se usan para validar que el sistema cumple los requisitos no funcionales de respuesta y estabilidad bajo carga.

Escenario	Usuarios simulados	Duracion	Criterio de exito
Carga normal del POS	50 usuarios concurrentes	5 min	P95 < 500ms, 0% errores
Pico de ventas	200 usuarios concurrentes	2 min	P95 < 1s, < 1% errores
Generacion de reportes	20 usuarios concurrentes	5 min	P95 < 3s, 0% errores
Prediccion ML bajo demanda	10 usuarios concurrentes	3 min	P95 < 5s, 0% errores
Estres del sistema completo	500 usuarios concurrentes	10 min	Degradacion controlada

4. AMBIENTE DE PRUEBAS

El sistema de pruebas es completamente aislado del entorno de produccion y se ejecuta dentro de contenedores Docker para garantizar reproducibilidad.

4.1 Infraestructura de testing

Servicio	Imagen Docker	Puerto	Uso en pruebas
Backend test	python:3.11-slim	8001	Instancia dedicada de FastAPI para tests
PostgreSQL test	postgres:15-alpine	5433	Base de datos aislada, se resetea entre suites
Redis test	redis:7-alpine	6380	Cache de pruebas, se flushea entre tests
Playwright	mcr.microsoft.com/playwright	--	Tests E2E en Chromium, Firefox, WebKit
Locust	locustio/locust	8089	Simulacion de carga

4.2 Datos de prueba

Dataset	Descripcion	Generacion
Tenant de prueba A	Dulceria ficticia con productos, ventas e inventario completo	Factory + fixtures pytest
Tenant de prueba B	Segundo tenant para validar aislamiento multi-tenant	Factory + fixtures pytest
Usuarios de prueba	Un usuario por cada rol: admin, vendedor, operador, supervisor	Factory pytest
Historial de ventas	365 dias de ventas simuladas para entrenamiento del modelo ML	Script generador
Productos con alertas	Productos con stock en minimo para disparar alertas	Fixtures pytest

```
# Estructura de fixtures pytest tests/   conftest.py          # Fixtures
globales: db, client, tokens   factories/     tenant_factory.py
user_factory.py     product_factory.py     sale_factory.py   unit/
integration/   e2e/
```

5. HERRAMIENTAS DE TESTING

Herramienta	Version	Uso	Capa
pytest	>=7.4	Framework principal de pruebas Python	Unitaria / Integracion
pytest-cov	>=4.1	Reporte de cobertura de codigo	Todas
pytest-asyncio	>=0.21	Soporte para pruebas de funciones async/await	Integracion
testcontainers	>=3.7	Levanta PostgreSQL y Redis reales en Docker para tests	Integracion
httpx	>=0.25	Cliente HTTP asincrono para testear la API FastAPI	Integracion
factory-boy	>=3.3	Generacion de datos de prueba con factories	Todas
faker	>=20.0	Datos ficticios realistas para fixtures	Todas
Jest	>=29.0	Framework de pruebas para componentes React	Frontend
React Testing Library	>=14	Pruebas de componentes orientadas al usuario	Frontend
Playwright	>=1.40	Pruebas E2E en multiples navegadores	E2E
Locust	>=2.17	Pruebas de carga y rendimiento	Rendimiento
OWASP ZAP	>=2.14	Escaneo de vulnerabilidades de seguridad	Seguridad
Bandit	>=1.7	Analisis estatico de seguridad en codigo Python	Seguridad
coverage.py	>=7.3	Medicion de cobertura de codigo	Cobertura
GitHub Actions	N/A	CI/CD: ejecucion automatica de pruebas en cada commit/PR	CI/CD

5.1 Configuracion de cobertura (pytest)

```
# pytest.ini [pytest] testpaths = tests asyncio_mode = auto addopts = --cov=app
--cov-report=html --cov-report=term --cov-fail-under=80  # .coveragerc [run]
source = app omit = app/migrations/*, tests/*  [report] fail_under = 80
show_missing = True
```

6. CASOS DE PRUEBA

6.1 Modulo de Autenticacion y Seguridad

ID	Caso de prueba	Entrada	Resultado esperado	Prio.
TC-AUTH-001	Login con credenciales validas	Email y password correctos	HTTP 200, JWT token valido	Alta
TC-AUTH-002	Login con password incorrecto	Email valido, password erroneo	HTTP 401, mensaje de error generico	Alta
TC-AUTH-003	Token JWT expirado	Token con exp en el pasado	HTTP 401, 'Token expirado'	Alta
TC-AUTH-004	Token JWT de otro tenant	JWT valido pero de Tenant B	HTTP 403 al acceder a datos de Tenant A	Critico
TC-AUTH-005	Acceso sin token	Request sin header Authorization	HTTP 401	Alta
TC-AUTH-006	Rate limiting login	5+ intentos fallidos en 1 minuto	HTTP 429, bloqueo temporal de IP	Alta
TC-AUTH-007	Acceso de vendedor a ruta de admin	JWT de rol vendedor a /admin/*	HTTP 403	Alta
TC-AUTH-008	Refresh token valido	Refresh token vigente	HTTP 200, nuevo access token	Media

6.2 Modulo POS - Registro de Ventas

ID	Caso de prueba	Entrada	Resultado esperado	Prio.
TC-POS-001	Registrar venta exitosa	Producto con stock, cantidad valida	HTTP 201, stock decrementado, recibo generado	Critico
TC-POS-002	Venta con stock insuficiente	Cantidad > stock disponible	HTTP 422, 'Stock insuficiente'	Alta
TC-POS-003	Venta con producto inexistente	producto_id no existente	HTTP 404	Alta
TC-POS-004	Venta con cantidad cero	cantidad = 0	HTTP 422, error de validacion	Alta
TC-POS-005	Venta dispara alerta de stock min.	Venta que lleva stock al minimo	Stock = minimo, alerta creada en sistema	Alta
TC-POS-006	Venta multi-producto	Carrito con 5 productos distintos	HTTP 201, cada producto descontado correctamente	Alta

TC-POS-007	Consultar detalle de venta	GET /ventas/{id} con token valido	HTTP 200, objeto venta con detalles completos	Media
TC-POS-008	Vendedor no ve ventas de otro tenant	JWT Tenant A, get ventas Tenant B	HTTP 403 o resultado vacio	Critico

6.3 Modulo de Inventario

ID	Caso de prueba	Resultado esperado	Prio.
TC-INV-001	Consultar stock de un producto	HTTP 200, cantidad actual correcta	Alta
TC-INV-002	Registrar entrada de lote	HTTP 201, stock incrementado, lote persistido	Alta
TC-INV-003	Lote con fecha de vencimiento pasada	HTTP 422, 'Fecha de vencimiento invalida'	Alta
TC-INV-004	Consultar productos con stock bajo minimo	HTTP 200, lista de productos en alerta	Media
TC-INV-005	Consultar lotes proximos a vencer (7 dias)	HTTP 200, lista filtrada correctamente	Media
TC-INV-006	Aislamiento inventario multi-tenant	Tenant A no ve inventario de Tenant B	Critico

6.4 Modulo de Reportes y Prediccion

ID	Caso de prueba	Resultado esperado	Prio.
TC-REP-001	Reporte de ventas por rango de fechas	HTTP 200, datos filtrados correctamente	Alta
TC-REP-002	Reporte con rango de fechas invalido	HTTP 422, 'Fecha inicio > fecha fin'	Media
TC-REP-003	KPIs del dashboard principal	HTTP 200, ventas del dia, semana y mes correctas	Alta
TC-REP-004	Prediccion de demanda de un producto	HTTP 200, array de forecast para 30 dias	Alta
TC-REP-005	Prediccion sin historial suficiente	HTTP 422 o respuesta con advertencia	Media
TC-BOT-001	Consulta: 'Cuanto vendi ayer'	HTTP 200, total de ventas de ayer en COP	Alta
TC-BOT-002	Consulta: 'Que productos tienen poco stock'	HTTP 200, lista de productos bajo minimo	Alta

TC-BOT-003	Consulta sin sentido de negocio	HTTP 200, mensaje de no comprension amigable	Media

7. METRICAS DE CALIDAD

Las siguientes metricas son obligatorias para aprobar el pipeline de CI/CD y realizar un release a produccion.

7.1 Metricas de cobertura

Metrica	Objetivo	Obligatorio para release
Cobertura total del backend	>= 80%	Si
Cobertura de capa de dominio	>= 90%	Si
Cobertura de casos de uso	>= 85%	Si
Cobertura de endpoints API	>= 80%	Si
Cobertura frontend (componentes)	>= 70%	Recomendado
Casos de prueba criticos ejecutados	100%	Si - Bloqueante
Casos de prueba de alta prioridad	>= 95%	Si

7.2 Metricas de rendimiento

Endpoint / Operacion	P50	P95	P99	Max permitido
POST /api/v1/ventas	< 100ms	< 300ms	< 500ms	1s
GET /api/v1/inventario	< 80ms	< 200ms	< 400ms	800ms
GET /api/v1/reportes/ventas	< 500ms	< 2s	< 3s	5s
GET /api/v1/predicciones	< 1s	< 3s	< 5s	8s
POST /api/v1/bot/consultar	< 500ms	< 2s	< 4s	6s
POST /api/v1/auth/login	< 200ms	< 500ms	< 800ms	1s

7.3 Metricas de defectos

Metrica	Objetivo
Defectos criticos abiertos al momento del release	0
Defectos bloqueantes abiertos al momento del release	0
Defectos de alta prioridad pendientes	< 3 (con workaround documentado)
Tasa de reapertura de defectos	< 5%

Tiempo promedio de resolucion de defecto critico	< 24 horas

8. GESTION DE DEFECTOS

8.1 Clasificacion de severidad

Severidad	Descripcion	SLA de resolucion	Ejemplos
Critico (S1)	El sistema es inutilizable o hay fuga de datos	< 4 horas	Fuga de datos multi-tenant, sistema caido, perdida de transacciones
Alto (S2)	Funcionalidad principal no funciona sin workaround	< 24 horas	POS no registra ventas, login bloqueado, inventario no actualiza
Medio (S3)	Funcionalidad limitada con workaround disponible	< 72 horas	Reporte con datos parcialmente incorrectos, bot sin respuesta
Bajo (S4)	Problema visual o de usabilidad sin impacto en operaciones	< 2 semanas	Texto mal alineado, color incorrecto, tooltip faltante

8.2 Ciclo de vida de un defecto

```
  [Nuevo]       |   [Asignado]  <- QA asigna al desarrollador responsable      |
[En progreso] <- Desarrollador trabaja en la correccion      |   [En revision]
<- PR creado, code review en curso      |   [Listo para retesting] <- Fix
mergeado al ambiente de QA      |   [Cerrado] <--- PASS    [Reabierto] <---
FAIL
```

8.3 Informacion requerida en un reporte de defecto

- ID unico del defecto (ej: BUG-2026-0042)
- Titulo descriptivo del problema
- Modulo y endpoint afectado
- Pasos exactos para reproducir el defecto
- Resultado esperado vs resultado actual
- Severidad y prioridad
- Evidencia: capturas de pantalla, logs, payload de la request
- Ambiente donde se reprodujo (testing / staging)
- Version del sistema afectada

9. AUTOMATIZACION Y CI/CD

Todas las pruebas automatizadas se ejecutan dentro del pipeline de GitHub Actions en cada commit y pull request. Un PR no puede mergearse si alguna etapa del pipeline falla.

9.1 Pipeline de pruebas automatizadas

```
# .github/workflows/tests.yml on: [push, pull_request]  jobs:   backend-tests:
steps:       - Lint (flake8, black, isort)       - Pruebas unitarias (pytest --
cov, fail-under=80)       - Pruebas de integracion (docker-compose up tests)
- Analisis de seguridad (bandit)       - Publicar reporte de cobertura
frontend-tests:     steps:       - Jest + React Testing Library       -
TypeScript type-check    e2e-tests: (solo en PR a main)     steps:       -
Playwright en Chromium + Firefox
```

9.2 Reglas de merge

Evento	Pruebas requeridas para merge
Commit a feature/*	Lint + Pruebas unitarias
PR a develop	Lint + Unitarias + Integracion + Cobertura >= 80%
PR a main (release)	Todo lo anterior + E2E + Seguridad + 0 defectos criticos
Hotfix a main	Pruebas de regresion del modulo afectado + E2E criticos

10. CRITERIOS DE ACEPTACION DEL SISTEMA

Para que el sistema SGDD sea considerado apto para produccion, TODOS los siguientes criterios deben cumplirse:

#	Criterio	Obligatorio
1	Cobertura de codigo del backend >= 80%	Si - Bloqueante
2	0 defectos de severidad Critica (S1) o Alta (S2) sin resolver	Si - Bloqueante
3	100% de casos de prueba marcados como Criticos ejecutados con resultado PASS	Si - Bloqueante
4	Aislamiento multi-tenant validado: ningun tenant accede a datos de otro	Si - Bloqueante
5	P95 de latencia del POS < 500ms bajo 50 usuarios concurrentes	Si - Bloqueante
6	Todos los flujos E2E criticos ejecutados en Playwright con resultado PASS	Si - Bloqueante
7	Escaneo OWASP ZAP completado sin vulnerabilidades de riesgo Alto o Critico	Si
8	Reporte de cobertura publicado y revisado por el equipo	Si
9	Pipeline de CI/CD ejecutado exitosamente en el commit del release	Si
10	Documentacion de defectos conocidos (si aplica) con workaround documentado	Si

Un release a produccion queda BLOQUEADO automaticamente si cualquier criterio marcado como 'Bloqueante' no se cumple. No existen excepciones a esta regla.

Estado documental del proyecto

Documento	Estado
SRS - Especificacion de Requisitos	Completo
SAD - Arquitectura de Software	Completo
SDD - Diseno de Software	Completo
ADR - Decisiones Arquitectonicas	Completo
STP - Plan de Pruebas (este documento)	Completo